Embedded Systems II

EEE3096/5S Final Exam 2020

Solutions

Section A

A.1. (b)

A.2. (b)

A.3. (a)

A.4. (d)

A.5. (a)

A.6. (b)

A.7. (c)

A.8. 7, 6, 1, 8

A.9. (d)

A.10. (c)

Illustration for A.8:

Acronym	→	#	Explanation
SBC		7	Single Board Computer
GPIO		6	General Purpose Input Output
SOC		1	System On Chip
ISO		8	International Organization for Standardization

Section B

B1.1. (a)

Could add comments or rename these inputs such as:

a → a_startnew (i.e. if a is high it sets the started register to true)
b → b_countto (i.e. b is the value to count up to)
c → c_reached (i.e. c is actually an output indicating the b values is reached)

B1.2. [FALSE]

because it is only influenced by the value of rst not the clock line.

B1.3. [FALSE]

because the lines ‘ x <= x + 1;’ and ‘if (x == b) c <= 1'b1;’ are non-blocking and moreover likely to execute simultaneously (according to standard Verilog) the value of x will be the same at the start of the x + 1 operation and the x == b operation, i.e. they run together and the updated value of applying the <= operation will only happen after the comparison is completed. So it should work reliably even for b = 0.

B1.4 (c)

B1.5 (a)

[I’ve put many as (a) - the options will be randomaized on vula]

B2.1

Any 3:

- Include headers: Include directly the code in stdio.h
- Expand macros: Expand and directly place the color definition strings of red and blue into the functions print_red and print_blue
- Evaluate macros: Determine if DEBUG has been defined and if so keep line 21 and if not remove it
- Remove comments

B2.2

-g Generate debugger tree/info so gdb/valgrind can be run on this code

-D DEBUG Defines DEBUG so that line 21 prints

-Wall prints all std warnings

-o colourprinter specifies the name of the output executable object file

B2.3

EEE3096S //In red [1/2 mark for correct string]
2020 //In blue [1/2 mark for correct string]
This is a debug message in main.c at 21 [1 mark for correct string showing 'main.c' and '21']

B2.4

Any 3

- Because it's open source it can be customised to bring in custom modules/drivers
- Because it's modular it can be optimised to contain only what is needed saving memory (it can be made very small)
- Because it's modular it can be optimised for low power use
- It can be configured to support real time operation
- It's fault tolerant by design
- It's openness combined with its maturity means there's a vast array of resources: drivers/kernel modules/ports/community support ecosystems/build tools
- Because it's standardised it is relatively easy to build portable code

B2.5

Temporal locality: The temporal locality principle states that if a data location is referenced then there is a high probability that it will be accessed again soon. [1]

Spatial locality: The spatial locality principle states that if a data location is referenced, then there is a high probability that data locations with nearby addresses will be accessed soon. [1]

Cache hierarchies exploit the above to place the data most likely to next be required by the processor in the fastest memory locations closest to the processor. Spatial and temporal locality mean the memory manager is able to make reliable predictions regarding what the next data to be needed will be. [2]

Section C

C.1

6 possible ways to get (at least) 4 marks

User and kernel space refer to different levels of privilege [1] that govern what hardware resources are accessible to processes running in each [1]. Applications/processes that run in user space operate with less privileged than those run in kernel space and must make use of system calls (part of the kernel) [1] to access hardware resources such as interfaces or memory [1].

The kernel is responsible for managing the use of hardware resources by scheduling [1] processes running in user space such as a browser, or your code [1].

C.2

7 possible ways to get 5 marks
Threads are lightweight [1] processes that occupy a single address space[1], can be swapped out of the CPU pipeline in a single cycle [1] with the program counter and register state all stored in dedicated support hardware [1] so it can be swapped back in later.

Processes are a software [1] level of parallelism consisting of multiple threads [1], and using potentially multiple address spaces [1], and which cost thousands of cycles to swap out[1].

C.3

Describe any 2, worth 2.5marks each
Pipelining: Takes advantage of parallelism inherent in the steps required to execute an instruction. By staging the process into multiple stages (eg Fetch, Decode, Execute, Memory Access, Writeback) the hardware used to carry out each stage can be in use every clock cycle on each consecutive instruction rather than sitting idle waiting for an instruction to fully complete before work on the next one can begin. This costs only additional hardware to handle hazards.

Multi-issue pipelines: A CPU pipeline can be duplicated in hardware (multiple ALUs added with additional control and data lines to feed them and handle the additional hazards). This system allows for multiple CPU instructions to be issued in parallel.

SIMD/Vector processing units: In scenarios where the same instruction is to be applied to many data points, it can be efficient to use dedicated hardware units that are optimised for applying the same operation to many pairs of operands simultaneously. Because the same instruction is being applied, while multiple ALU units are required full duplication of the control and data path is not required making SIMD/VPUs more hardware efficient than simply implementing wider ,multi-issue CPUs. Additionally, SIMD/VPUs are supported by vector register files making memory accesses significantly more efficient (1 memory

read/write's worth of clock cycles operates on a vector's worth of operands in parallel). Additionally these memory accesses will make efficient use of of caching given the spatial locality such data exhibit.

Hardware multithreading (MIMD): For the cost of additional hardware to store the state of a CPU pipeline (Register file state, instruction, and program counter) and increase complexity of scheduling (for fine grained and simultaneous multithreading) it is possible for a CPU to seamlessly switch between operating on multiple different sequences of instructions (threads) for the cost of a single cycle. Switching between threads allows for optimal use of the CPU pipeline and maximise use of the full issue width available by scheduling whichever thread of instructions is able to make the most use of available hardware or even by interweaving threads.

Multi-core (MIMD): Multiple CPU cores can be placed on a single silicon die allowing a programmer to explicitly write parallel code. The programmer must explicitly write multithreaded/multi-process code to take advantage of these multiple cores. To make this task easier commonly a shared memory space is assumed (all processes have access to the same memory) a programmer must therefore take care to not overwrite memory locations used by other processes.

Loop unrolling: Both compilers and CPU schedulers take advantage of this technique. By unrolling a loop into its constituent CPU instructions (or pipeline operation stages) it is possible to find and exploit low levels of parallelism executing stages of execution as soon as it is logically possible and the hardware available as opposed to waiting for each loop iteration to complete in full before any work is done on the next iteration.

C.4

It is **(c)** an real-time embedded system. As it has real-time characteristics, sampling data in real time that needs to be responded to within certain deadlines.

C.5

A safety-critical system is one whose failure or inability to meet deadlines could result in serious problems such as: serious injury or death to people or loss or severe damage to important property or environment. There is no clear indication of how this system may lead to such severe consequences if it fails … the worse case scenario is perhaps that there's a big Earthquake - which the user someone doesn't observe - which could lead to the user not taking suitable precautions and getting damaged… but this is guessing, there is no specified safety-critical aspects (e.g. injury or damage) that may result if the system fails. Therefore there is no mitigating reason indicaitng this must be considered safety-critical.

A hard deadline must be met (the operation done) within a prescribed timelimit otherwise the system is considered to have failed (in some serious way, e.g. potential harm to people). Hard deadlines are more common in safety-critical … no clearly indicated harddeadline has

been specified in the NatureSens system description. A soft deadline, on the other hand, is a task that needs to be completed withing a particular time, but failure to do so does not mean the whole system fails catastrophically ... the NatureSens has lots of soft deadlines.

C.6

a) It's a Mealy Machine for sure (like them better because they suggest a more responsive system where the output may activate immediately on the transition, rather than waiting until the state starts).
b) The hierarchical feature of state charts would be very useful in this case, being able to described always operation as a concurrent set of states that operates in parallel to the other modes.
c) The student may want to add additional states so as to provide a accurate representation, but this is not so necessary, can be completed just be adding additional transitions with suitable input triggers and output activations on the edges.

C.7

This was intended as a bit of a tricky one to end with, so that those students thinking hard and working effectively will get a slightly high score.
Basically, for this the main problem is the busy wait, it's inaccurate; supposedly the author calculated how long pieces of the code took to run based on instructions being executed (as in concerning to ASM and then figuring out how fast the processor would go through it). But it didn't fully account for the outer loop, so longer delays will get more and more inaccurate (not that this actually mattners, because long delays will probably not get used). This code would essentially steal clockcycles from the processor to do delays which could have otherwise been either handled by system calls (which would release the CPU to do other things / tasks) or use of an I2C hardware controller module would have offloaded most of this work to such a device instead of having the CPU do the work. If it was used in sampling the ADC as well, it would probably be terribly inefficient.